

INTERVIEW STUDY GUIDE & PROJECT REFERENCE

Healthcare Portal

React Deep Dive

A complete technical breakdown of every hook, pattern, workflow, and concept used in the healthcare frontend project — written to help you confidently explain it in any interview.

STACK

React + Redux Toolkit + Vite

PAGES

20+ Role-based Pages

NEW HOOKS

6 Beyond Your Knowledge

TABLE OF CONTENTS

1. Project Overview & Architecture
2. Folder Structure Explained
3. React Concepts You Already Know
4. NEW React Concepts Introduced in This Project
5. Redux Toolkit — Deep Dive
6. React Router — Every Pattern Used
7. React Context API — Toast System

8. The Full Appointment Workflow (End-to-End)

9. How Each Page Works

10. Interview Q&A Prep

1 Project Overview & Architecture

This is a **multi-role healthcare management portal** built with React. It simulates the real-world workflow of a clinic:

WHAT DOES THIS APP DO?

- **Patients** can search for doctors, book appointments, view billing, and receive notifications.
- **Doctors** can view their appointment queue, complete consultations, message patients, and issue certificates.
- **Admins** can approve/reject appointment requests, view all clinic activity, and manage users.

Tech Stack

LAYER	TECHNOLOGY	WHY IT'S USED
UI Framework	React 18	Component-based SPA (Single Page App)
Build Tool	Vite	Fast dev server, instant HMR (Hot Module Replace)
State Management	Redux Toolkit	Shared global state for appointments, billing, auth, notifications
Routing	React Router v6	Navigation between pages without page reload
Styling	Tailwind CSS	Utility-first CSS classes for fast UI building
Data	JSON files (dummy data)	Simulates a backend/database (no real API)

Data Flow Architecture

User Action (click button)



Component calls dispatch(action)



Redux Slice reducer updates state



useSelector() re-reads updated state



React re-renders the component automatically

2 Folder Structure Explained

```
src/
├── components/      ← Reusable UI pieces used across pages
│   ├── Layout.jsx   ← Wraps every page (Sidebar + content)
│   ├── Sidebar.jsx  ← Left nav, role-aware links
│   ├── Navbar.jsx   ← Top bar with user info
│   └── Toast.jsx     ← Global notification toasts
├── pages/           ← Each page = one .jsx file
│   ├── auth/        ← Login, Register, Unauthorized
│   ├── patient/     ← Dashboard, BookAppointment, Billing...
│   ├── doctor/      ← Dashboard, Appointments, Messages...
│   └── admin/       ← Dashboard, ManageUsers, Schedules...
├── redux/
│   ├── store.js      ← Combines all reducers
│   └── slices/       ← Each slice = one feature's state
│       ├── authSlice.js
│       ├── appointmentSlice.js
│       ├── billingSlice.js
│       ├── notificationSlice.js
│       ├── recordsSlice.js
│       └── patientSlice.js
├── routes/
│   ├── AppRoutes.jsx ← ALL route definitions
│   └── ProtectedRoute.jsx ← Auth + role guard
├── data/             ← Static JSON files (fake database)
│   ├── dummyUsers.json
│   ├── dummyDoctors.json
│   ├── dummyPatients.json
│   ├── dummyAppointments.json
│   └── dummyBills.json
└── main.jsx          ← Entry point, wraps app in providers
```

 INTERVIEW TIP

When asked "explain your folder structure" — say: *"I separated concerns by feature area. Components are reusable, pages are one per route, slices handle isolated state domains, and data is separate from logic. This makes scaling and debugging easy."*

3 React Concepts You Already Know

Here's how each concept you know is actually used in this project:

useState

REACT HOOK

Stores local component data that changes over time. Used in almost every page.

Example from DoctorSearch.jsx:

```
const [searchText, setSearchText] = useState("");  
const [selectedSpecialization, setSelectedSpecialization] = useState("");  
const [showSuggestions, setShowSuggestions] = useState(false);
```

Logic: When the user types, `setSearchText` is called → state updates → React re-renders the filtered doctors list automatically.

useEffect

REACT HOOK

Runs code *after* the component renders. Used for side-effects like event listeners, API calls, timers.

Example from DoctorSearch.jsx — Close dropdown on outside click:

```
useEffect(() => {  
  const handleClickOutside = (event) => {  
    if (searchRef.current && !searchRef.current.contains(event.target)) {  
      setShowSuggestions(false);  
    }  
  };  
  
  document.addEventListener("mousedown", handleClickOutside);  
  
  return () => {  
    document.removeEventListener("mousedown", handleClickOutside);  
  };  
}, []); ← empty array = runs only once on mount
```

The cleanup function (`return () => ...`) removes the event listener when the component unmounts, preventing memory leaks.

Link / NavLink

REACT ROUTER

Navigate between routes without a full page reload. `NavLink` adds an active class automatically when the URL matches.

Sidebar.jsx uses NavLink:

```
<NavLink  
  to="/patient/my-appointments"  
  className={({ isActive }) =>  
    isActive ? "bg-teal-500 text-white" : "text-slate-400 hover:text-white"  
  }  
>  
  My Appointments  
</NavLink>
```

Key difference: `Link` is a plain link. `NavLink` knows if it is the current active route.

Routes / Route

REACT ROUTER

AppRoutes.jsx:

```
<Routes>  
  <Route path="/" element={ <DemoHome /> } />  
  <Route path="/patient/dashboard" element={  
    <ProtectedRoute allowedRoles={["patient"]} >  
      <PatientDashboard />  
    </ProtectedRoute>  
  } />  
</Routes>
```

`Routes` looks at the current URL and renders the matching `Route`'s element.

4 NEW React Concepts in This Project

These are the hooks and patterns **beyond your current knowledge** — study these carefully as they come up in every senior interview.

`useSelector`

REDUX HOOK — NEW

What it does: Reads (subscribes to) a piece of data from the Redux store. Every time that slice of data changes, the component automatically re-renders.

Syntax:

```
const variableName = useSelector((state) => state.sliceName.propertyName);
```

Real usage in this project:

```
// In MyAppointments.jsx
const auth = useSelector((state) => state.auth);
const appointments = useSelector((state) => state.appointment?.appointments || []);
```

THINK OF IT LIKE THIS

`useSelector` is like "subscribing to a newspaper section." When that section updates, you automatically get the new edition. You don't fetch it manually — React + Redux does it for you.

useDispatch

REDUX HOOK — NEW

What it does: Returns the `dispatch` function. You call `dispatch(action)` to send a command to the Redux store to update state.

Pattern:

```
const dispatch = useDispatch();  
  
// When user clicks a button:  
dispatch(cancelAppointment(appointment.id));
```

Full flow in MyAppointments.jsx:

```
const dispatch = useDispatch();  
  
const handleCancel = (appointment) => {  
  dispatch(cancelAppointment(appointment.id));  
  // Redux store updates → useSelector re-reads → UI updates  
};
```

USESELECTOR VS USEDISPATCH

`useSelector` = READ from store. `useDispatch` = WRITE to store.

useNavigate

REACT ROUTER HOOK — NEW

What it does: Gives you a function to *programmatically* navigate to a route — useful after form submit, button click, or any logic-driven redirect.

Usage in BookAppointment.jsx:

```
const navigate = useNavigate();

const handleSubmit = (e) => {
  e.preventDefault();
  dispatch(addAppointment(newAppointment));
  navigate("/patient/my-appointments"); ← go to page after submit
};
```

Difference from Link: `Link` = clickable element. `useNavigate` = navigate from inside JavaScript logic.

useParams

REACT ROUTER HOOK — NEW

What it does: Extracts dynamic URL parameters from the current route path.

Route definition in AppRoutes.jsx:

```
<Route path="/patient/book-appointment/" ... />
```

Then in BookAppointment.jsx:

```
const { doctorId } = useParams();
// If URL is /patient/book-appointment/2, then doctorId = "2"

const doctor = doctors.find((d) => d.id === parseInt(doctorId));
```

The colon `:` in the route path marks it as a variable segment that `useParams` can read.

useRef

REACT HOOK — NEW

What it does: Creates a mutable reference to a DOM element or any value that **persists across re-renders without causing a re-render** when it changes.

Used in DoctorSearch.jsx to detect outside clicks:

```
const searchRef = useRef(null);

// Attach to a DOM element:
<div ref={searchRef} > ← React puts the real DOM node here

// Use it in logic:
if (searchRef.current && !searchRef.current.contains(event.target)) {
  setShowSuggestions(false);
}
```

USESTATE VS USEREF

- `useState` — changes trigger a re-render
- `useRef` — changes do NOT trigger a re-render; it just holds a reference

useCallback**REACT HOOK — NEW**

What it does: Memoizes (caches) a function so it isn't recreated on every re-render. Used for performance optimization and stable function references.

Used in Toast.jsx:

```
const removeToast = useCallback((id) => {  
  setToasts((prev) => prev.filter((t) => t.id !== id));  
}, []); ← empty array = same function instance forever  
  
const addToast = useCallback((message, type, title, duration) => {  
  const id = Date.now() + Math.random();  
  setToasts((prev) => [...prev, { id, message, type, title, duration }]);  
}, []);
```

WHY USE IT HERE?

If `removeToast` was a plain function, it would be a new object on every render. Since it's passed as a prop to child `ToastItem`, the child would re-render unnecessarily.

`useCallback` keeps it stable.

Navigate (component)

REACT ROUTER — NEW

What it does: Renders a redirect as JSX — when React renders `<Navigate to="/login" />`, it immediately redirects the user.

Used in ProtectedRoute.jsx:

```
if (!isAuthenticated) {  
  return <Navigate to="/login" replace />;  
}  
// replace = don't add to browser history (no back button loop)
```

Difference from useNavigate: `Navigate` is a JSX element (used inside return). `useNavigate` is a hook (used inside functions/event handlers).

5

Redux Toolkit — Deep Dive

Redux is used for **global state** — data that multiple components across different pages need to share and update.

The 3 Core Building Blocks

CONCEPT	WHAT IT IS	WHERE IT LIVES
Store	The single source of truth — holds all state	<code>redux/store.js</code>
Slice	A feature's state + its reducers bundled together	<code>redux/slices/*.js</code>
Reducer	A pure function that takes state + action and returns new state	Inside each slice
Action	An object describing what happened: <code>{type: "...", payload: ...}</code>	Auto-created by <code>createSlice</code>

How a Slice is Built — `appointmentSlice.js`


```
import { createSlice } from "@reduxjs/toolkit";
import appointmentsData from "../../data/dummyAppointments.json";

const appointmentSlice = createSlice({
  name: "appointment",          ← slice name (used in store)
  initialState: {
    appointments: appointmentsData ← starting data from JSON file
  },
  reducers: {
    addAppointment: (state, action) => {
      state.appointments.push(action.payload);
    },
    cancelAppointment: (state, action) => {
      const appt = state.appointments.find((a) => a.id === action.payload);
      if (appt) appt.status = "cancelled";
    },
    completeAppointment: (state, action) => {
      const appt = state.appointments.find((a) => a.id === action.payload);
      if (appt) {
        appt.status = "completed";
        appt.completedAt = new Date().toISOString().split("T")[0];
      }
    }
  }
});

export const { addAppointment, cancelAppointment, completeAppointment } = appointmentSlice
export default appointmentSlice.reducer;
```

IMMER UNDER THE HOOD

Redux Toolkit uses a library called **Immer** internally. That's why you can write

`state.appointments.push(...)` — normally mutating state directly is illegal in Redux. Immer wraps it safely and creates a new state object behind the scenes.

Registering All Slices in store.js

```
import { configureStore } from "@reduxjs/toolkit";
import authReducer from "../slices/authSlice";
import appointmentReducer from "../slices/appointmentSlice";
import billingReducer from "../slices/billingSlice";
import notificationReducer from "../slices/notificationSlice";

export const store = configureStore({
  reducer: {
    auth: authReducer,
    appointment: appointmentReducer,
    billing: billingReducer,
    notifications: notificationReducer
  }
});
```

Connecting the Store to React in main.jsx

```
import { Provider } from "react-redux";
import { store } from "../redux/store";

createRoot(document.getElementById("root")).render(
  <Provider store={store}> ← makes store available to ALL components
    <App />
  </Provider>
);
```

All 6 Slices in This Project

SLICE	STATE IT MANAGES	KEY ACTIONS
authSlice	Logged-in user info, role, isAuthenticated	login, logout
appointmentSlice	All appointments array	addAppointment, cancelAppointment, approveAppointment, rejectAppointment, completeAppointment
billingSlice	Patient bills array	addBill, payBill

<code>notificationSlice</code>	Notification messages	<code>addNotification</code> , <code>markRead</code> , <code>markAllRead</code>
<code>recordsSlice</code>	Medical records / EHR	<code>addRecord</code> , <code>updateRecord</code>
<code>patientSlice</code>	Patient profiles	<code>updatePatient</code>

6 React Router — Every Pattern Used

Pattern 1: BrowserRouter (set up in App.jsx)

```
import { BrowserRouter } from "react-router-dom";
```

```
<BrowserRouter>
```

```
<AppRoutes />
```

```
</BrowserRouter>
```

BrowserRouter enables HTML5 history-based routing. The URL changes without page reloads.

Pattern 2: Dynamic Routes with :param

```
// Route definition
```

```
<Route path="/patient/book-appointment/:doctorId" element={...} />
```

```
// Navigate to it with the ID
```

```
navigate(`/patient/book-appointment/${doctor.id}`);
```

```
// Extract it in the destination page
```

```
const { doctorId } = useParams();
```

Pattern 3: Protected Route (Custom Guard Component)

```
function ProtectedRoute({ children, allowedRoles }) {
  const { isAuthenticated, user } = useSelector((state) => state.auth);

  if (!isAuthenticated) return <Navigate to="/login" replace />;
  if (allowedRoles && !allowedRoles.includes(user.role))
    return <Navigate to="/unauthorized" replace />;

  return children;
}

// Usage – wraps any route that needs protection:
<Route path="/admin/dashboard" element={
  <ProtectedRoute allowedRoles={["admin"]} >
    <AdminDashboard />
  </ProtectedRoute>
} />
```

HOW PROTECTEDROUTE WORKS — STEP BY STEP

1. React tries to render the route's element
2. ProtectedRoute reads the Redux auth state
3. If not logged in → redirects to `/login` instead of rendering children
4. If wrong role → redirects to `/unauthorized`
5. Only if both checks pass → renders the actual page (`children`)

Pattern 4: NavLink with Active Class

```
<NavLink  
  to="/patient/dashboard"  
  className={({ isActive }) =>  
    `flex items-center gap-3 px-3 py-2.5 rounded-xl transition  
    ${isActive  
      ? "bg-teal-500/20 text-teal-300 font-bold"  
      : "text-slate-400 hover:text-white"  
    }`  
  }  
  >  
    Dashboard  
  </NavLink>
```

`NavLink` passes an `isActive` boolean into the `className` function, letting you style the active link differently.

7 React Context API — Toast System

The Toast (popup notification) system uses the **Context API** — a built-in React way to share data globally without passing props through every component layer.

Why Context? The Problem it Solves

WITHOUT CONTEXT — PROP DRILLING

*App → Layout → Page → Button → needs toast()
You'd have to pass "toast" as a prop through every layer = messy*

WITH CONTEXT — DIRECT ACCESS

Any component anywhere → const toast = useToast() → done!

How it's Built in Toast.jsx

Step 1: Create the context

```
const ToastContext = createContext(null);
```

Step 2: Create a hook to consume it

```
export function useToast() {  
  return useContext(ToastContext);  
}
```

Step 3: Create the Provider that holds the state

```
export function ToastProvider({ children }) {  
  const [toasts, setToasts] = useState([]);  
  
  const toast = {  
    success: (msg) => addToast(msg, "success"),  
    error: (msg) => addToast(msg, "error"),  
    ...  
  };  
  
  return (  
    <ToastContext.Provider value={toast} >  
      {children}  
    </ToastContext.Provider>  
  );  
}
```

How to Use it in Any Component


```
import { useToast } from "../../components/Toast";

function MyComponent() {
  const toast = useToast();

  const handleSave = () => {
    toast.success("Record saved successfully!", "Saved");
    toast.error("Something went wrong", "Error");
    toast.warning("Slot is almost full");
  };
}
```

8

The Full Appointment Workflow (End-to-End)

This is the most important logic to explain in an interview. Walk the interviewer through this step-by-step:

1

Patient Searches for a Doctor — DoctorSearch.jsx

Patient types in search box → `useState` tracks the text → doctors JSON is filtered in real-time using `.filter()` → results rendered as cards. Clicking a doctor calls `navigate(`/patient/book-appointment/${doctor.id}`)`.

2

Patient Books Appointment — BookAppointment.jsx

`useParams()` extracts `doctorId` from the URL → doctor data loaded → patient selects date + time + reason → on submit: `dispatch(addAppointment({...}))` → appointment added to Redux store with `status: "pending"`. Toast shows success.

3

Admin Approves — ManageSchedules.jsx

Admin sees all pending appointments from Redux state via `useSelector`. Clicks "Approve" → `dispatch(approveAppointment(id))` → status becomes `"confirmed"` → `dispatch(addNotification(...))` sends real-time notification to patient.

4

Doctor Marks Complete — DoctorAppointments.jsx

Doctor sees only *their own* confirmed appointments (filtered by `doctorId`). Clicks "Mark Complete" → dispatches 3 actions in sequence: `completeAppointment(id)` + `addBill({...})` + `addNotification({...})`. Bill auto-generated. Patient notified.

5

Patient Pays Bill — PatientBilling.jsx

Patient sees the new bill from `useSelector(state => state.billing.bills)`. Selects payment method → `dispatch(payBill({billId, paymentMethod}))` → bill status changes to `"paid"`.

THE 3 DISPATCHES ON MARK COMPLETE — MEMORIZE THIS!

```
dispatch(completeAppointment(appointment.id)); ← updates appointmentSlice  
dispatch(addBill({ patientId, doctorId, ... })); ← updates billingSlice  
dispatch(addNotification({ title, message, ... })); ← updates notificationSlice
```

One user action (button click) → three slice updates → all connected components re-render automatically.

9 How Each Page Works

Patient Pages

PAGE	KEY LOGIC	HOOKS/CONCEPTS USED
PatientDashboard	Shows summary stats (upcoming appointments, unpaid bills). Reads from Redux.	useSelector, useNavigate
DoctorSearch	Filters doctor list by name/specialization. Shows dropdown suggestions. Closes on outside click.	useState, useEffect, useRef, useNavigate
BookAppointment	Loads doctor from URL param. Checks slot capacity. Dispatches new appointment to Redux.	useParams, useSelector, useDispatch, useNavigate, useToast
MyAppointments	Filters all appointments by current patient's ID. Cancel button dispatches status change.	useSelector, useDispatch, useToast
PatientBilling	Shows all bills for this patient. Pay button opens modal with payment method selection.	useSelector, useDispatch
Notifications	Reads from notificationSlice. Click marks as read. "Mark all read" dispatches to Redux.	useSelector, useDispatch

Doctor Pages

PAGE	KEY LOGIC	HOOKS/CONCEPTS USED
DoctorDashboard	Shows only this doctor's appointments (filtered by doctorId). Quick action links.	useSelector
DoctorAppointments	Filtered queue. "Mark Complete" triggers 3 dispatches: complete + bill + notification.	useSelector, useDispatch, useToast

DoctorMessages	Local conversation state. Select patient = see thread. Send = adds message to local state.	useState
IssueCertificate	Doctor selects patient + certificate type + dates → generates printable preview in same page.	useState, useSelector
DoctorSchedule	Weekly slot grid. Toggle day on/off updates local state array.	useState

Admin Pages

PAGE	KEY LOGIC	HOOKS/CONCEPTS USED
AdminDashboard	Aggregated stats across all slices (total patients, appointments, revenue).	useSelector
ManageSchedules	Shows pending + all appointments. Approve/reject dispatches status change + notification.	useSelector, useDispatch, useToast
AdminAppointments	Full searchable, filterable table of all appointments. Approve/reject from this view too.	useState, useSelector, useDispatch
ManageUsers	Lists all users. Reads from dummyUsers JSON. Role badge display.	useState

10 Interview Q&A Prep

These are real questions interviewers ask for React + Redux roles. Memorize the logic, not just the words.

Q. What is the difference between local state (useState) and global state (Redux)?

A. Local state with `useState` lives inside a single component and is lost when the component unmounts. It's used for UI-only data like form inputs, toggle states, or dropdown visibility. Global state with Redux lives in the store, persists across navigation, and is accessible by any component in the tree. In this project, appointment data is in Redux because both the patient, doctor, and admin pages all read and update it. A form input like the search text box is just `useState` because no other page needs it.

Q. How does the appointment booking flow work end-to-end?

A. The patient navigates to DoctorSearch, clicks a doctor which pushes the doctor's ID into the URL (`/book-appointment/:doctorId`). BookAppointment reads that ID with `useParams()` , loads the doctor from JSON, and shows available time slots. On submit, it dispatches `addAppointment()` to the Redux store with status "pending". The admin then sees this in ManageSchedules via `useSelector` , clicks Approve which dispatches `approveAppointment()` and `addNotification()` . The doctor sees it in their queue, marks it complete which dispatches `completeAppointment()` , `addBill()` , and another notification. The patient then sees the bill in PatientBilling and pays it with `payBill()` .

Q. What is ProtectedRoute and how does it work?

A. ProtectedRoute is a custom wrapper component that reads the auth slice from Redux using `useSelector` . If the user is not authenticated, it returns `<Navigate to="/login" />` instead of the children, immediately redirecting. If the user is authenticated but has the wrong role (e.g., a patient trying to access an admin route), it redirects to `/unauthorized` . Only when both checks pass does it render `children` — the actual page. It's used by wrapping every route in AppRoutes with it, passing the allowed roles as a prop.

Q. What is the Context API and why did you use it for Toast instead of Redux?

A. The Context API is React's built-in way to share data globally without prop drilling. I used it for the Toast system because toast notifications are UI-only ephemeral state — they appear for 4 seconds and disappear. They don't need to be persisted in the Redux store, synced with the backend, or shared after the toast disappears. Redux is heavier and best suited for persistent application data. Context is lighter and perfect for transient UI state. The pattern is: create a context, wrap the app in a Provider, and any component calls `useToast()` to get the toast functions.

Q. What is useRef and when should you use it instead of useState?

A. Use `useRef` when you need to access a DOM element directly or store a value that should persist between renders without triggering a re-render. In this project, I used it in DoctorSearch to reference the search container div. When the user clicks anywhere outside that div, the `mousedown` event fires and I check `searchRef.current.contains(event.target)` to decide if the click was inside or outside. If I had used `useState` for this, updating it would cause a re-render. `useRef` keeps it as a stable reference with no re-render cost.

Q. What is useCallback and when is it needed?

A. `useCallback` memoizes a function — it returns the same function instance between renders unless the dependencies change. It's used in the Toast system for `removeToast` and `addToast` because these functions are passed as props to child components. Without `useCallback`, every re-render of ToastProvider would create a new function reference, causing every ToastItem child to re-render unnecessarily. With `useCallback` and an empty dependency array, the function stays the same reference forever.

Q. What is the difference between useNavigate and the Navigate component?

A. `useNavigate` is a hook that returns a function you call imperatively from inside event handlers or logic: `navigate("/path")`. You use it when navigation happens as a result of some logic — like after a form submit. `<Navigate />` is a JSX component that causes an immediate redirect when rendered. You use it in conditional rendering — like in ProtectedRoute where you return `<Navigate to="/login" />` if the user is not logged in. Think of it as: hook = use in functions, component = use in return JSX.

Q. How does Redux Toolkit's createSlice differ from traditional Redux?

A. Traditional Redux requires you to write action type constants (`const ADD_APPOINTMENT = "ADD_APPOINTMENT"`), action creator functions, and a separate reducer with a switch statement — all in different files. Redux Toolkit's `createSlice` combines all of this into one object. It auto-generates action creators from the reducer function names, uses Immer internally so you can write mutating syntax, and exports both the reducer and actions from the same file. It reduces boilerplate by about 70%.

Q. How does the notification system work in this project?

A. There is a `notificationSlice` in Redux with an array of notification objects. Each object has a `userId` to link it to the correct patient. When an appointment is approved by admin, the code dispatches `addNotification({title, message, type, userId})` — this pushes a new notification into the slice. The patient's Notifications page uses `useSelector` to read all notifications and filters them by the current user's ID. When the patient clicks a notification, `markRead(id)` is dispatched, setting `unread: false` for that item, which removes the unread dot indicator.

Key Concepts Summary — Quick Reference

HOOK / CONCEPT	FROM	WHAT IT DOES	WHEN YOU USE IT
<code>useState</code>	React	Local component state	Any UI data that changes
<code>useEffect</code>	React	Side effects after render	Event listeners, timers, fetch
<code>useRef</code>	React	DOM reference / stable value	Outside click, focus, timers
<code>useCallback</code>	React	Memoize a function	Prevent child re-renders
<code>useContext</code>	React	Read from Context	Global UI state (toast, theme)
<code>createContext</code>	React	Create a context object	Define a shared data channel
<code>useSelector</code>	react-redux	Read from Redux store	Any component needing global state

<code>useDispatch</code>	react-redux	Dispatch actions to store	Update global state
<code>createSlice</code>	@reduxjs/toolkit	Define state + reducers	Each feature's state management
<code>configureStore</code>	@reduxjs/toolkit	Combine all slices	App entry point (store.js)
<code>useNavigate</code>	react-router-dom	Navigate programmatically	After form submit / logic
<code>useParams</code>	react-router-dom	Get URL params	Dynamic routes like :doctorId
<code>Link</code>	react-router-dom	Client-side anchor	Static nav links
<code>NavLink</code>	react-router-dom	Link with active class	Sidebar/nav with active state
<code>Navigate</code>	react-router-dom	Redirect in JSX	ProtectedRoute, auth guards
<code>Routes/Route</code>	react-router-dom	Render components by URL	App router setup



FINAL INTERVIEW TIP

When explaining this project, use this 3-part formula for any feature:

1. **What the user sees** — "The admin clicks Approve on a pending appointment"
2. **What happens in code** — "It calls `dispatch(approveAppointment(id))` and `dispatch(addNotification(...))` "
3. **What the user sees next** — "The appointment status immediately changes to confirmed in the UI, and the patient gets a notification"